ChatGPT

Comparing Vibe Coding to Traditional Outsourced Development

Vibe Coding in Context: A New Name for an Old Approach

Vibe coding refers to the practice of describing a desired software outcome in natural language (the "vibe" or vision of what you want) and having code produced to match that description, typically via AI code generation. In essence, vibe coding is *specification-driven development* with a tight feedback loop: you define what you want, see it implemented, give feedback, and iterate – all **without manually writing or even reading all the code** in detail. In a real sense, software teams have *always* engaged in a form of vibe coding; the difference now is the **speed and efficiency** with which the loop closes, thanks to generative AI.

In the past, a business stakeholder or technical lead would write a brief or user story ("this is what we want, here's the flow or feature needed"), hand it to developers, and later review the resulting software's behavior against expectations. That stakeholder typically wasn't reviewing the source code line by line – they only cared that the software *felt* right and met the requirements. This process – describe, implement, observe, refine – **is vibe coding in spirit**, just executed by human intermediaries over days or weeks instead of seconds or minutes. As one industry article notes, *"most software developers don't write every line of code from scratch… they draw on existing libraries… to get basic components in place,"* which speeds up development but can reduce visibility into the code [1]. Vibe coding supercharges this idea by using AI to generate those components on the fly based on a high-level brief, giving stakeholders a new level of immediacy in seeing their ideas come to life.

The Business Perspective: Faster Feedback and Control

From a **business user's perspective**, vibe coding offers a level of control and rapid experimentation that traditional outsourcing or large internal dev teams struggled to provide. In the past, business stakeholders would detail requirements up front (often imperfectly), then wait as a distant team implemented them. By the time a feature was delivered for review, weeks or months might have passed, and the result often missed the mark – partly because *what the business thought it wanted wasn't exactly right, or was misunderstood*. Changing direction meant another lengthy cycle. With vibe coding, that cycle compresses dramatically. Business users can **interact with an AI pair programmer in real-time**, tweaking the "vibe" (requirements) and seeing changes almost immediately. This tight feedback loop is essentially Agile development on steroids – enabling iterative refinement in hours or days instead of quarters.

Importantly, this approach **democratizes the development feedback loop**. Non-technical stakeholders no longer feel as "locked out" of the development process waiting for technical experts to decode their vision. In vibe coding sessions, a product owner can say "I don't like how this behaves, let's change it" and watch the AI adjust the code accordingly. In the past, such feedback would be given in meetings or tickets and then filtered through layers of project managers and developers. Now the **business side** gains a more hands-on role in shaping the software, without needing to write code themselves. This leads to a sense of empowerment – they aren't just handing off a spec into a black box, but actively steering the outcome. Many organizations have struggled historically with this gap: for

example, poor communication and misaligned expectations between business clients and outsourced dev teams contribute to **57% of IT projects failing**, and *62% of outsourced IT projects running over budget* [2]. Vibe coding's tight, interactive loop directly addresses these communication failures by keeping the stakeholder engaged throughout development.

Traditional Outsourcing: Slow "Vibe Coding" with Human Developers

When we talk about **outsourced software development**, we're essentially describing a slower, more indirect form of vibe coding. In a classic outsourcing scenario – say hiring an offshore development team or contracting a third-party agency – the dynamic is: you **describe what you want** (in requirements documents, user stories, or design meetings), the external team writes the code, then they deliver the software for your review. You typically evaluate the delivered product (maybe via demos or test environments) and give feedback, **without personally reading the code**. Sound familiar? It's the same *define → implement → evaluate* loop, just stretched over much longer timelines and with humans in the middle.

The key differences lie in speed, alignment, and transparency. Traditional outsourcing often suffers from communication lags and misunderstandings due to time zone differences, language/cultural gaps, and the sheer overhead of coordinating teams. Studies have found that miscommunication leads to *70% longer timelines* on outsourced projects on average [3]. It's no wonder that *56% of project failures* in outsourced development are attributed to communication breakdowns [2]. By contrast, vibe coding with an AI model can be almost instantaneous – you prompt the AI and get an answer in seconds. There's little ambiguity in what you "asked" versus what gets "built," since you see output right away and can correct course. The feedback loop that might take two weeks and a series of meetings with an external team can occur in an afternoon with an AI assistant.

Alignment of incentives is another big contrast. An outsourced vendor is a business in their own right – their goal is to satisfy the contract and perhaps maximize billable hours or lock in ongoing work. Your goal, as the client, is to get the best solution to your problem. These goals don't always perfectly align. If requirements were unclear, an outsourcing firm might implement something literal to the spec even if it doesn't truly solve the underlying problem, because that's what was "asked for." In worse cases, outsourcing partners may not proactively point out a better solution because it's not in scope (or they plan to charge change orders). Lack of alignment and shared vision is cited as the number one reason outsourcing engagements fail – *"it will still fail if they don't align with your mission and vision"* as one CTO put it [4]. Vibe coding, in contrast, eliminates the separate business entity with its own motivations. The "outsourced developer" here is an AI model that has no agenda other than to follow your instructions. Of course, the onus is then on *you* (or your in-house engineers) to provide clear instructions and steer the development – but you no longer need to worry about hidden motives or corners cut for convenience.

Quality oversight also shifts in interesting ways. With outsourced teams (or even large internal teams), a project lead might find it *impossible to review every line of code* produced – especially if managing 5, 10, or 20 developers. In practice, leaders review design docs and maybe some critical sections, but much of the code is taken on faith until issues arise. Vibe coding doesn't automatically guarantee quality, but it does mean the **one implementing the code (the AI) is effectively working under your constant supervision**. You can inspect the AI's output each step of the way. If something looks off, you catch it immediately and adjust the prompt or code. In essence, vibe coding makes the development process more observable to the requester: rather than waiting for a big code drop, you see code as it's being produced. However, *human oversight remains crucial*. As Alex Zenla (CTO of a cloud security firm) noted,

AI can generate insecure or low-quality code if its training data contained bad patterns, reintroducing vulnerabilities that human developers had already learned to avoid [5]. Thus, a skilled engineer guiding the AI (and reviewing its drafts) is key to achieving high-quality results.

Open-Source Libraries: The Original Outsourced Codebase

Long before AI-driven vibe coding, software developers commonly "outsourced" development work by leveraging **open-source libraries** and third-party components. Instead of reinventing the wheel, a developer might pull in a popular library to handle, say, authentication, logging, or data parsing. In doing so, they implicitly trust code written by *someone else (often an open-source community)* to become part of their application. This approach is enormously productive – it's the foundation of modern software. In fact, **70–90% of code in a modern application comes from open-source components rather than in-house code** [6]. Just as vibe coding allows developers to not write every line themselves, open-source usage has meant we rarely write everything from scratch either [1].

However, relying on third-party packages is a double-edged sword. It introduces a **dependency supply chain** – essentially an external codebase now embedded in yours – which can carry risks and downsides:

- **Bloat and Unneeded Functionality:** A library designed to be general-purpose often includes many features you don't actually use. By importing it, you bring along this extra baggage. For example, if you only needed a simple logging functionality, including a mega-library might pull in advanced features (and related code) you never touch. Many developers have been shocked to find that adding one small npm package can balloon into **hundreds of dependent packages** in `node_modules`. Even *"small random projects can have over a whopping 1000 dependencies"* once all those transitive packages are included [7]. This is effectively adding thousands of lines of someone else's code – and potential bugs – that your team isn't intimately familiar with.

- **Security Vulnerabilities:** When you import a library, you also import its vulnerabilities. If that library (or one of its deep dependencies) has a known security flaw, your software now has that flaw. Alarmingly, **86% of codebases contain at least one known open-source vulnerability** according to a 2025 analysis, and 81% contain a high- or critical-severity vulnerability [8]. Often this happens because projects pull in components and never update them – the same report found the average application has ~911 open-source dependencies and *90% of those are outdated by over 4 years* [9] [10]. A notorious example was the *Log4j* library's **Log4Shell** flaw in late 2021: a widely used logging library had an obscure feature that allowed loading data from external sources (JNDI lookups), which attackers turned into a severe remote code execution exploit [11] [12]. Countless applications were vulnerable *simply because they included Log4j* without realizing this hidden "feature." If a developer had instead written a minimal logging utility for their needs (or nowadays, asked an AI to generate one), they likely wouldn't have spontaneously added such an exotic capability. In other words, including third-party code can introduce **functionality you never asked for**, some of which might undermine your security or stability.

- **Malicious or Unreliable Dependencies:** The open-source ecosystem, especially package managers like npm or PyPI, has seen incidents of **supply chain attacks** – where a maintainer's account is hacked or a popular package is subverted to include malicious code. For instance, in one case attackers compromised an npm maintainer and injected credential-stealing malware into extremely common packages, which then propagated automatically into millions of downstream projects [13]. When you "outsource" via open source, you are entrusting not just the original authors but the entire chain of trust behind those packages. If any link is compromised,

your software can be poisoned. Without rigorous monitoring, these risks often go unnoticed until damage is done, because no one is reviewing every line of code in those dependencies.

- **License and Maintenance Issues:** Using third-party code means complying with its open-source license (which can have legal implications) and relying on the maintainers to keep it updated. Some projects become abandoned or don't promptly fix discovered bugs, leaving you stuck or forcing you to expend effort to fork/maintain it yourself. As the footprint of others' code in your product grows, so does the complexity of keeping track of licenses and updates (hence the recent push for SBOMs – Software Bills of Materials – to track these components [14]). In effect, by outsourcing a chunk of your software to open source, you also outsource part of your ongoing maintenance, for better or worse.

In summary, open-source libraries have been an invaluable form of "outsourced development" – much like vibe coding aims to be – delivering huge productivity gains by leveraging existing code. But this comes at the cost of importing external assumptions, potential vulnerabilities, and loss of some control over that part of your codebase. Vibe coding is now emerging as an *alternative way to get code you didn't write yourself*, which invites a comparison: **Is AI-generated code any better or worse than open-source code?**

Vibe Coding vs. Third-Party Libraries: A Head-to-Head Comparison

Let's directly compare using **AI-generated code (vibe coding)** to using **third-party libraries or outsourced human code** for implementing a feature/module. In many ways, these are analogous – all three approaches involve someone else writing the initial code (an AI model, an open-source community, or an external dev team). The differences lie in what you get and how you manage it:

- 🏗️ **Scope & Bloat:** Vibe coding can produce a *tailor-made solution* that includes exactly what you specify – no more, no less. You have the ability to constrain the AI: for example, "generate a logging module that does X and Y, and nothing fancy beyond that." The output will generally be focused on your described needs. In contrast, importing a popular library for that purpose might bring along a ton of extra functionality you don't use. As noted, it's common for a single library to drag in many transitive dependencies and thousands of lines of code [7]. All that extra code represents things that could go wrong (or need updating). With vibe coding, **the code footprint is often smaller** because it's generated to spec. (Of course, one must be careful – AI models might sometimes pull in a dependency by habit. A savvy developer can explicitly instruct, "do not use external libraries unless necessary" to keep the solution self-contained.)

- 🔒 **Security & Trust:** Neither approach is inherently 100% secure – vigilance is required in both cases – but they offer different profiles of risk. Using a third-party component means trusting that external code, which could harbor known vulnerabilities or even malicious logic. We saw that a huge proportion of codebases contain known vulnerable components [8]. If you didn't write that code, you might not notice a dangerous function call or insecure default. With AI-generated code, *you have the opportunity (and responsibility) to review the code immediately*. The AI isn't guaranteed to produce secure code – in fact, if it was trained on insecure examples, it might reproduce those bugs [5]. Security researchers warn that *"AI is its own worst enemy in terms of generating code that's insecure"* if used naively [5]. However, the big advantage is **transparency**: you can see the code and run your security scans on it the moment it's generated. There's no opaque binary or obfuscated third-party bundle – it's your code now. A diligent developer can leverage this: for instance, run static analysis or have the AI explain any tricky part of its output.

In essence, vibe coding **shifts security left** – you treat the AI like a junior developer whose work must be reviewed. By explicitly prompting for secure practices (e.g., "use safe coding patterns, handle edge cases, include input validation") you can guide the AI, and then verify the result. With open source, you often only find out about a vulnerability when an advisory gets published later. That said, one must also be cautious: AI code may introduce new types of supply-chain risk if it *auto-includes libraries or snippets* you didn't realize. For example, researchers have noted AI may suggest libraries that have known CVEs (if training data was outdated) or even hallucinate package names that don't exist, potentially tricking developers into pulling malicious look-alike packages [15]. So the takeaway is: **with vibe coding you trade blind trust for the need to actively review and test AI output** – a trade that, in the hands of a skilled engineer, can lead to a more secure outcome because you're inspecting what you use.

- ⚙ **Maintainability & Ownership:** Code generated via vibe coding is *owned by you* from day one. It lives in your codebase, and your team can modify or extend it as needed. This is a big shift from using a third-party module, where you typically treat that module as a black box (upgrading it when the maintainer releases a new version, or patching it only in emergencies). Many developers have experienced frustration when a library doesn't do *exactly* what they need, or has a bug that's not fixed quickly – you either hack around it or fork the library (which is a heavy burden because now you become the maintainer). With AI-generated code, you essentially **fork at inception** – the code is purpose-built for you, and you're free to adapt it. Over time, this can mean *lower technical debt*, because the module can evolve with your project. You can refactor it to match your coding standards, simplify it further, or optimize it for your specific use case. You're not at the mercy of an outside maintainer's roadmap. Additionally, because you likely prompted the AI with your **preferred architecture and style**, the output might already be aligned with your broader system design (for example, you can instruct the AI to use your project's logging interface or follow your naming conventions). This coherence makes the code easier for your team to understand and maintain. By contrast, plugging in an external library often introduces a new "foreign" style or requires writing adapters to make it fit your patterns.

- ⏱ **Speed of Iteration:** Both open-source integration and vibe coding are faster than writing code from scratch, but their iteration speeds differ. If an open-source library doesn't do exactly what you hoped, you might spend days evaluating alternatives or combing through docs to tweak behavior. With vibe coding, if the first AI-generated solution isn't right, you can immediately refine the prompt or edit the code and ask the AI to adjust the implementation. The iteration cycle is extremely tight – often a matter of minutes to regenerate a section of code. This means you can experiment and converge on a solution much faster. It's akin to having a super-speed pair programmer who can rewrite the module in 30 seconds when you say "actually, let's try a different approach." This agility is something traditional outsourcing cannot match, and even open-source libraries (which might have slow release cycles) can't compete with. Essentially, **vibe coding enables a trial-and-error approach at low cost**: you're no longer stuck with the first solution. If you pulled in a library and found it heavy or unsuitable, switching it out later can be a big project; but if an AI-generated code isn't up to par, you can just regenerate or modify it in-line.

- **Focus and Custom Features:** An underrated aspect of vibe coding is that you can ask for exactly the features you need, and *not implement things you don't need*. This sounds obvious, but consider how many third-party solutions come with configuration options and features that you might never use. Every extra feature is also extra complexity and attack surface. For instance, if you had asked an AI to create a logging utility, you likely wouldn't include something like *"allow remote class loading of log message formats"* (which was the root of the Log4j fiasco). And if the AI bizarrely did include something beyond your spec, it would stand out in code review as a non-

sequitur (a kind of AI hallucination to be removed). In contrast, when using a general-purpose library, you often don't even realize it has esoteric features – until they bite you. By keeping development focused on what's needed, vibe coding helps maintain **a cleaner, tighter codebase**. Additionally, you can request customizations that a generic library wouldn't have. Want your logging to output in a particular JSON structure to fit your monitoring tool? You can prompt the AI to include that. With a library, you might be stuck with its format or have to add conversion layers. In short, vibe coding offers *bespoke code on demand*, whereas traditional outsourcing or libraries often require you to bend your needs to what the off-the-shelf solution can do.

It's worth noting that **vibe coding is not a panacea**. The ease of generating code means one can also generate a lot of *poor* code quickly if not careful. As with any outsourcing (human or library), strong technical leadership and governance are needed. Best practices emerging in the industry include treating AI like a junior dev – use linters, tests, and code reviews as you normally would. In fact, you can even have the AI assist in writing unit tests for its own code (a common prompt pattern) to ensure the module behaves as expected. Only 18% of organizations say they have a formal list of approved tools or policies for vibe coding, even though AI-generated code now constitutes a significant portion of codebases in many companies [16]. This shows that governance is still catching up to the technology. In regulated or security-sensitive environments, one might need even stricter checks (e.g. scanning AI outputs for secrets or known vulnerable patterns, as security researchers suggest [17]).

The Developer's Role: Guided AI vs. Blind Outsourcing

One theme that emerges in comparing vibe coding to past outsourcing models is the **central importance of the developer in the loop**. Vibe coding is most powerful in the hands of a developer or engineer who *truly understands coding and architecture*. This individual essentially plays the role of **architect, code reviewer, and prompt engineer**. They translate the business "vibe" into precise prompts, enforce quality standards, and integrate the AI's output into the larger system. The value of vibe coding isn't that it replaces developers – it's that it **amplifies** them. A great developer can produce even more with an AI assistant, while maintaining oversight.

By contrast, consider an outsourced project where a non-technical founder just hands everything to an external team: the outcome is entirely dependent on that team's expertise and diligence, with the founder having little ability to assess code quality. Vibe coding shifts that dynamic – the person prompting the AI can (and should) be technical enough to know what good code looks like. They're not reading *every single line* perhaps, but they are formulating the structure and skimming the output for sanity. They can ask the AI *why* it chose a certain approach, or to document the code for clarity. This is akin to a tech lead guiding a junior programmer: *the lead provides direction and reviews the result, but doesn't manually type every character*.

The **bottom line** is that vibe coding should be seen as a new form of outsourcing where **control remains with your internal team**. You are "outsourcing" the grunt work of typing code to an AI, but not outsourcing the thinking or responsibility. In the past, if you outsourced unwisely, you might end up with a pile of low-quality code that your in-house folks then struggle to take over (or, worst case, a failed project requiring a rewrite). With vibe coding, any low-quality draft can be immediately iterated on, or simply not merged until it meets the bar – since you have the steering wheel. One security researcher in WIRED described AI-generated code as a rough draft that *"may not fully take into account all specific context… the process relies on human reviewers' ability to spot every possible flaw or incongruity"* [18]. This underscores that the human-in-the-loop is still responsible for the final quality. The advantage is that

this rough draft is produced very quickly, giving humans more time to polish and less time on boilerplate.

Conclusion: Evolution of Outsourcing – From Offshore to Open-Source to AI

"Vibe coding" is, in many respects, **the latest evolution in outsourcing software development**. First we had outsourcing to external teams (contracting work out to people you don't directly supervise). Then we had outsourcing to open-source libraries (delegating functionality to code you didn't write). Now we have outsourcing to AI models that can generate code on demand. Each evolution aimed to make development faster and more efficient by leveraging external help – whether human or machine – and each comes with its own challenges in ensuring quality and security.

The comparison highlights that **vibe coding isn't an alien concept dropped into software engineering out of nowhere**. It builds on the same principles of modularity and abstraction that we've used for decades. The big leap is the immediacy and flexibility it offers. When done with care, vibe coding can combine the best aspects of prior outsourcing models while mitigating some of their worst flaws. You get the *speed and lower cost* of not writing everything yourself (like open source gives), but you also retain much more *control and alignment* with your specific needs, since you literally dictate the code's intent to the AI. There's no contract to negotiate or community to persuade – the AI will try its best to give you what you asked for. If what it gives you isn't right, you can immediately ask again in a different way. This fluidity is unprecedented in earlier outsourcing paradigms.

Of course, with great power comes great responsibility: **the speed of vibe coding can amplify mistakes** just as it accelerates features. It's now possible to introduce a bug or security issue into multiple parts of a codebase extremely quickly if one isn't careful (just as one could if copy-pasting Stack Overflow solutions recklessly). The **human element – expertise, review, testing – remains critical**. In fact, one could argue it's even more critical: since AI can produce 1000 lines of code in the blink of an eye, teams must be vigilant in reviewing those lines, just as they would (or should) with an outsourced contractor's delivery. The encouraging news is that all the knowledge and tools we've developed to manage outsourced and open-source code (code reviews, automated tests, static analyzers, dependency audits, etc.) are directly applicable to AI-generated code. Many of the best practices for vibe coding simply involve *applying these controls proactively*: e.g., **prompt the AI to adhere to your coding standards and include tests**, use sandbox environments to run and validate the code, and keep an eye on any libraries the AI is pulling in. As one LinkedIn tech commentator put it, *"treat the AI as a junior developer requiring mandatory review"* [19] – a succinct guideline that encapsulates how to approach vibe coding safely.

In comparing vibe coding to outsourcing models of the past, an "apples to apples" mindset is useful. We shouldn't compare AI-generated code to an imaginary ideal of perfectly handcrafted code with no bugs (because that's rarely the reality in any scenario). Instead, we compare it to the actual alternatives: **hiring outsiders or adding third-party packages**. By that measure, vibe coding in the hands of a skilled engineer can often produce a solution *as good as or better than* a typical outsourced outcome, delivered far faster and tuned more precisely to requirements. It reduces dependency bloat, since you generate only what you need, and it keeps the intellectual property (and responsibility) of the code within your team's grasp, rather than handing it off entirely. It's not magic – you're still going to invest time in refining prompts and verifying results – but it shifts the heavy lifting to a tireless assistant.

In summary, vibe coding represents a powerful new way to outsource the toil of coding without outsourcing your vision. It's the latest step in a long trend of increasing abstraction in software

development. Those who embrace it wisely – combining the **creativity of human developers** with the **productivity of AI** – are likely to build software faster and more flexibly than ever before. But like any outsourcing, success will depend on communication (in this case, prompts), oversight, and alignment of the final product with the true needs of the users. With these practices in place, vibe coding can indeed transform the software development workflow, succeeding where many traditional outsourcing projects have failed, and avoiding the pitfalls of blindly relying on massive third-party codebases.

Sources:

- Newman, L. H. (2025). *Vibe Coding Is the New Open Source—in the Worst Way Possible*. **WIRED** – *"Most developers don't write every line from scratch… draw on existing libraries… efficient but can create exposure and lack of visibility"* [1] ; *Security concerns with AI-generated code reintroducing old vulnerabilities* [5] ; *Human reviewers must catch flaws in AI code* [18] ; *Survey data on AI-generated code in organizations (60% of code by AI, few have policies)* [16] .
- AlterSquare (2025). *The True Cost of Poor Communication in Outsourced Development Projects – Statistics on outsourcing failures: 57% of IT projects fail due to communication issues; 62% of outsourced projects over budget; 70% timeline overruns* [2] [3] ; *50% of outsourced projects fail to meet expectations (30% due to client-vendor communication issues)* [20] .
- Magalhães, C. (2022). *12 Reasons Why Outsourcing Software Development Fails – Importance of alignment with outsourcing partner's vision; "will fail if they don't align with your mission and vision."* [4] .
- Stack Exchange (2025). *What percentage of software is open-source? – "70–90% of any given software code base is made up of open source components"* [6] , based on Linux Foundation and Synopsys studies.
- Alger, J. (2025). **Security Magazine** – *Open Source Security and Risk Analysis report: 86% of codebases had open-source vulnerabilities, 81% had high/critical vulns; average app has 911 open-source dependencies; 90% of components outdated >4 years* [8] [9] [10] .
- Syki (2023). *How Many Dependencies Does Your Project Really Have? – Even small projects can end up with 1000+ npm dependencies due to transitive packages* [7] .
- LinkedIn Post (Rob Vissers, 2025) – *Discussion of "Vibe Coding: The High-Speed Supply Chain Threat"*: highlights that AI-generated code often auto-includes libraries, creating a *"high-speed supply chain vulnerability machine"* if not checked; mentions recent npm supply-chain attack and other risks like dependency hallucination and insecure patterns [21] [17] .

---

[1] [5] [16] [18] Vibe Coding Is the New Open Source—in the Worst Way Possible | WIRED
https://www.wired.com/story/vibe-coding-is-the-new-open-source/

[2] [3] [20] The True Cost of Poor Communication in Outsourced Development Projects | by AlterSquare | Medium
https://altersquare.medium.com/the-true-cost-of-poor-communication-in-outsourced-development-projects-a9853e3b3a46

[4] 12 Reasons Why Outsourcing Software Development Fails in 2022
https://altar.io/10-reasons-why-outsourcing-software-development-fails/

[6] statistics - What percentage of all software is open-source? - Open Source Stack Exchange
https://opensource.stackexchange.com/questions/15461/what-percentage-of-all-software-is-open-source

[7] How Many Dependencies Does Your Project Really Have? - DEV Community
https://dev.to/syki/how-many-dependencies-does-your-project-really-have-239c

[8] [9] [10] [14] Open source software vulnerabilities found in 86% of codebases | Security Magazine
https://www.securitymagazine.com/articles/101420-open-source-software-vulnerabilities-found-in-86-of-codebases

11 12 Log4j vulnerability explained: What is Log4Shell?
https://www.dynatrace.com/news/blog/what-is-log4shell/

13 15 17 19 21 AI Code Generation vs. | Peter Winston
https://www.linkedin.com/posts/peterwinston_ai-cybersecurity-softwaredevelopment-activity-7302830817666322432-8JYx